

Lab 8 mmap 实验报告

李睿阳 524120910059

一、实验目标与整体设计

本实验在 xv6 中实现 `mmap` 和 `munmap`，使用户程序可以把文件映射到进程虚拟地址空间，并通过普通内存访问文件内容。

实现采用 lazy allocation：

- `mmap` 只记录映射关系，不立即分配物理页。
- 第一次访问映射区时触发 page fault。
- page fault handler 根据 VMA 找到对应文件和偏移，分配物理页并读入文件内容。
- `munmap` 解除映射，必要时把 `MAP_SHARED` 的 dirty page 写回文件。
- `fork` 复制 VMA 元数据并增加文件引用计数。
- `exit` 清理仍然存在的映射。

二、设计 VMA 数据结构

`mmap` 的核心是记录虚拟地址区间和文件区间之间的映射关系。每个进程维护固定大小的 VMA 数组，每个 VMA 保存：

- `valid`：是否使用。
- `va`：映射起始虚拟地址。
- `length`：映射长度，按页向上取整。
- `prot`：访问权限。
- `flags`：映射类型。
- `f`：映射文件。
- `offset`：文件偏移。

修改文件：

- `kernel/proc.h`

关键代码：

```
struct vma {  
    int valid;  
    uint64 va;  
    uint64 length;  
    int prot;  
    int flags;  
    struct file *f;  
    uint64 offset;  
};
```

在 struct proc 中加入：

```
struct vma vmas[16];
```

三、实现 mmap

mmap 负责创建 VMA，不负责实际分配物理页。

主要流程：

1. 读取系统调用参数。
2. 通过 argfd 获取文件对象。
3. 检查 PROT_WRITE | MAP_SHARED 是否满足文件可写条件。
4. 在 VMA 数组中寻找空闲项。
5. 选择映射虚拟地址。
6. 填写 VMA，并调用 filedup 增加文件引用。
7. 返回映射起始地址。

修改文件：

- kernel/sysfile.c

关键代码：

```

uint64
sys_mmap(void)
{
    uint64 addr;
    int length, prot, flags, fd;
    uint64 offset;
    struct file *f;
    struct proc *p = myproc();

    if(argfd(4, &fd, &f) < 0)
        return 0xffffffffffffffffL;

    argaddr(0, &addr);
    argint(1, &length);
    argint(2, &prot);
    argint(3, &flags);
    argaddr(5, &offset);

    if((prot & PROT_WRITE) && (f->writable == 0) && (flags & MAP_SHARED))
        return 0xffffffffffffffffL;

    struct vma *nvma = 0;
    for(int i = 0; i < 16; i++){
        if(p->vmass[i].valid == 0){
            nvma = &p->vmass[i];
            break;
        }
    }
    if(nvma == 0)
        return 0xffffffffffffffffL;

    uint64 va_start = 0x40000000;
    for(int i = 0; i < 16; i++){
        if(p->vmass[i].valid && p->vmass[i].va + p->vmass[i].length > va_start)
            va_start = PGROUNDUP(p->vmass[i].va + p->vmass[i].length);
    }

    nvma->valid = 1;
    nvma->va = va_start;
    nvma->length = PGROUNDUP(length);
    nvma->prot = prot;
    nvma->flags = flags;
    nvma->f = f;

```

```
nvma->offset = offset;

filedup(f);
return nvma->va;
}
```

四、处理 mmap lazy page fault

由于 mmap 不立即建立页表映射，访问映射区时会触发 page fault。内核需要判断 fault 地址是否属于某个 VMA，如果属于，则补充分配和映射。

主要流程：

1. 查找 fault 地址所在的 VMA。
2. 将 fault 地址向下对齐到页边界。
3. 分配并清零物理页。
4. 根据 VMA 起点和文件偏移计算读取位置。
5. 从文件读取一页内容。
6. 根据 prot 设置页表权限。
7. 调用 mappages 建立映射。

修改文件：

- kernel/trap.c

关键代码：

```

int
handle_mmap_pagefault(uint64 va)
{
    struct proc *p = myproc();
    struct vma *v = 0;

    if(va >= MAXVA || va == 0)
        return -1;

    for(int i = 0; i < 16; i++){
        if(p->vmas[i].valid &&
            va >= p->vmas[i].va &&
            va < p->vmas[i].va + p->vmas[i].length){
            v = &p->vmas[i];
            break;
        }
    }
    if(v == 0)
        return -1;

    uint64 page_va = PGROUNDDOWN(va);
    char *mem = kalloc();
    if(mem == 0)
        return -1;
    memset(mem, 0, PGSIZE);

    uint64 map_offset = page_va - v->va;
    uint64 file_offset = v->offset + map_offset;

    uint read_size = PGSIZE;
    if(map_offset + PGSIZE > v->length)
        read_size = v->length - map_offset;

    ilock(v->f->ip);
    int n = readi(v->f->ip, 0, (uint64)mem, file_offset, read_size);
    iunlock(v->f->ip);

    if(n < 0){
        kfree(mem);
        return -1;
    }

    int perm = PTE_U;

```

```

    if(v->prot & PROT_READ)
        perm |= PTE_R;
    if(v->prot & PROT_WRITE)
        perm |= PTE_W;

    if(mappages(p->pagetable, page_va, PGSIZE, (uint64)mem, perm) != 0){
        kfree(mem);
        return -1;
    }
    return 0;
}

```

usertrap 中同时处理 COW fault 和 mmap fault:

```

} else if(r_scause() == 13 || r_scause() == 15){
    uint64 va = r_stval();
    if(r_scause() == 15 && cow_handler(p->pagetable, va) == 0){
        ;
    } else if(handle_mmap_pagefault(va) < 0){
        p->killed = 1;
    }
}
}

```

五、实现 munmap

`munmap` 解除指定范围的映射。实验保证不会在 VMA 中间挖洞，因此只需要处理取消开头、结尾或整个 VMA。

主要流程：

1. 找到覆盖目标区间的 VMA。
2. 遍历目标区间内已经映射的页。
3. 如果是 `MAP_SHARED` 且 PTE dirty bit 被置位，则写回文件。
4. 通过 PTE 获取物理地址后调用 `writel`。
5. 调用 `uvmunmap` 解除页表映射并释放物理页。
6. 根据取消范围调整或清空 VMA。

修改文件：

- `kernel/sysfile.c`

关键代码：

```

int
do_munmap(uint64 addr, int length)
{
    struct proc *p = myproc();
    struct vma *v = 0;

    for(int i = 0; i < 16; i++){
        if(p->vmass[i].valid &&
            addr >= p->vmass[i].va &&
            addr + length <= p->vmass[i].va + p->vmass[i].length){
            v = &p->vmass[i];
            break;
        }
    }
    if(v == 0)
        return -1;

    for(uint64 a = addr; a < addr + length; a += PGSIZE){
        pte_t *pte = walk(p->pagetable, a, 0);
        if(pte && (*pte & PTE_V)){
            if((v->flags & MAP_SHARED) && (*pte & PTE_D)){
                uint64 file_offset = v->offset + a - v->va;
                uint64 map_offset = a - v->va;

                uint write_size = PGSIZE;
                if(map_offset + PGSIZE > v->length)
                    write_size = v->length - map_offset;

                uint64 pa = PTE2PA(*pte);
                begin_op();
                ilock(v->f->ip);
                writei(v->f->ip, 0, pa, file_offset, write_size);
                iunlock(v->f->ip);
                end_op();
            }
            uvmunmap(p->pagetable, a, 1, 1);
        }
    }

    if(addr == v->va && length == v->length){
        v->valid = 0;
        fclose(v->f);
    } else if(addr == v->va){

```

```

    v->va += length;
    v->offset += length;
    v->length -= length;
} else {
    v->length -= length;
}
return 0;
}

```

系统调用包装：

```

uint64
sys_munmap(void)
{
    uint64 addr;
    int length;
    argaddr(0, &addr);
    argint(1, &length);
    return do_munmap(addr, length);
}

```

六、进程退出时清理 VMA

进程退出时，需要清理仍未解除的 mmap 区域，避免文件修改丢失、物理页泄漏和文件引用泄漏。

修改文件：

- kernel/proc.c

关键代码：

```

extern int do_munmap(uint64, int);

for(int i = 0; i < 16; i++){
    if(p->vmass[i].valid)
        do_munmap(p->vmass[i].va, p->vmass[i].length);
}

```

七、fork 时复制 VMA

fork 后，子进程继承父进程的 mmap 区域。这里只复制 VMA 元数据，不复制实际映射页；子进程访问映射区时会重新触发 lazy page fault。

由于 VMA 中保存了 `struct file *`，复制后需要调用 `filedup` 增加文件引用计数。

修改文件：

- `kernel/proc.c`

关键代码：

```
for(int i = 0; i < 16; i++){
    if(p->vmass[i].valid){
        np->vmass[i] = p->vmass[i];
        filedup(p->vmass[i].f);
    }
}
```

八、处理历史 FS lab 修改的影响

运行 `usertests -q` 时，`writebig` 曾出现磁盘块不足。原因是之前的 FS lab 修改扩大了 `MAXFILE`，导致 `writebig` 写入规模过大。

修正方式是在 `LAB_FS` 下启用双重间接块规模，其他 lab 使用普通 xv6 的 `MAXFILE`。

修改文件：

- `kernel/fs.h`

关键代码：

```
#define NDIRECT 11
#define NINDIRECT (BSIZE / sizeof(uint))
#ifdef LAB_FS
#define MAXFILE (NDIRECT + NINDIRECT + NINDIRECT*NINDIRECT)
#else
#define MAXFILE (NDIRECT + NINDIRECT)
#endif
```

九、遇到的问题与解决方案

1. page fault 地址要向下对齐

问题：最初处理 page fault 时，如果直接使用 fault 地址或向上取整，可能会把错误的页映射到页表中，导致当前访问的页仍然缺页，进程继续触发 page fault。

原因：r_stval() 返回的是触发异常的具体虚拟地址，它可能位于页中间，而不是页起始地址。

解决方案：建立页表映射前，必须将 fault 地址向下对齐到页边界：

```
uint64 page_va = PGROUNDDOWN(va);
```

2. 写回文件要使用物理地址

问题：munmap 写回 dirty page 时，如果把用户虚拟地址直接传给 writei，会导致写回失败或写入错误数据。

原因：用户虚拟地址只在用户页表中有意义，内核不能直接把它当成自己的地址访问。

解决方案：先通过 walk 找到 PTE，再从 PTE 中取出物理地址传给 writei：

```
uint64 pa = PTE2PA(*pte);  
writei(v->f->ip, 0, pa, file_offset, write_size);
```

3. 部分 unmap 要同步更新 offset

问题：当 munmap 取消 VMA 开头部分后，如果只更新 va 和 length，后续再次 page fault 会从错误的文件位置读取内容。

原因：VMA 的虚拟起点后移后，它对应的文件偏移也应该同步后移，否则虚拟地址和文件偏移的对应关系会被破坏。

解决方案：取消 VMA 开头部分时，同时更新 va、offset 和 length：

```
v->va += length;  
v->offset += length;  
v->length -= length;
```

4. mmap fault 不能覆盖 COW fault

问题：该仓库之前实现过 COW。处理 store page fault 时，如果直接进入 mmap fault 逻辑，可能会破坏已有的 COW 处理。

原因：store page fault 既可能来自 COW 写时复制，也可能来自 mmap lazy allocation，两者需要区分处理。

解决方案：先尝试处理 COW fault；如果不是 COW，再尝试处理 mmap fault：

1. 先尝试 `cow_handler`。
2. 如果不是 COW，再尝试 `handle_mmap_pagefault`。
3. 两者都失败，则杀死进程。

5. 文件引用计数需要维护

问题：如果 mmap 后用户关闭了文件描述符，而 VMA 中仍然保存原来的 `struct file *`，后续 page fault 或 munmap 可能访问已经释放的文件对象。

原因：VMA 的生命周期可能长于文件描述符，不能依赖用户态 fd 保持打开。

解决方案：mmap 创建 VMA 时调用 `filedup(f)`，VMA 被完全解除时调用 `fileclose(v->f)`。

6. fork 后 VMA 要复制但页不必复制

问题：如果 fork 时不复制 VMA，子进程访问父进程已有的 mmap 区域会触发非法访问。

原因：mmap 映射属于进程地址空间的一部分，子进程应该继承这些映射关系。

解决方案：fork 时复制 VMA 元数据，并对其中的文件对象调用 `filedup`。实际物理页不需要复制，子进程访问时重新 lazy fault 即可。

7. exit 时需要清理残留映射

问题：如果进程退出前没有显式调用 `munmap`，可能造成 dirty page 未写回、物理页泄漏和文件引用泄漏。

原因：用户程序不一定会主动解除所有映射，内核必须在进程退出时兜底清理资源。

解决方案：在 `exit` 中遍历所有有效 VMA，对每个 VMA 调用 `do_munmap`。

8. read/write 长度要处理最后一页

问题：映射长度不一定正好是页大小的整数倍，最后一页可能只对应文件中的一部分内容。

原因：如果最后一页仍然按完整一页读写，可能越过当前映射范围，造成错误的数据读写。

解决方案：根据 `map_offset` 和 `v->length` 计算实际读写长度：

```
uint read_size = PGSIZE;
if(map_offset + PGSIZE > v->length)
    read_size = v->length - map_offset;
```

9. VMA 地址范围不能重叠

问题：如果新的 `mmap` 地址和已有 VMA 重叠，后续查找 VMA、page fault 和 `munmap` 都可能定位到错误的映射。

原因：同一个虚拟地址不能同时属于多个文件映射。

解决方案：为新 VMA 选择地址时，从固定基址开始向上查找，并避开已有 VMA 的范围：

```
uint64 va_start = 0x40000000;
for(int i = 0; i < 16; i++){
    if(p->vmass[i].valid && p->vmass[i].va + p->vmass[i].length > va_start)
        va_start = PGROUNDUP(p->vmass[i].va + p->vmass[i].length);
}
```